

Pseudo Code Objective Questions (Bitwise & Operators)

Question 1

Which bitwise operator is used to swap two numbers?

- A) Left Shift (<<)
- B) Right Shift (>>)
- C) Bitwise OR (|)
- D) Bitwise XOR (^)

Question 2

What is the output of the below code for input `num = 15`?

```
unsigned int fun(unsigned int n) {  
    unsigned int count = 0;  
    while (n) {  
        count += n & 1;  
        n >>= 1;  
    }  
    return count;  
}
```

- A) 15
- B) 5
- C) 4
- D) 3

Question 3

Which is the format specifier used to prefix `0x` and print a number in hexadecimal notation?

- A) `%O`
- B) `%#`
- C) `%x`
- D) `%#x`

Question 4

Which is the **Bit Toggling** operator?

- A) Bitwise OR (|)
- B) Bitwise XOR (^)
- C) Bitwise AND (&)
- D) Bitwise NOT / Tilde (~)

Question 5

What is the output of the below expression?

```
000111 | 01010
```

- A) 0000000
- B) 00
- C) 010111
- D) None

Question 6

Bitwise & can be used in conjunction with ~ operator to turn off 1 or more bits in a number.

- A) True
- B) False

Question 7

If we have to add two numbers without having any carry, which bitwise operator should be used?

- A) Bitwise OR (|)
- B) Bitwise AND (&)
- C) Bitwise NOT (~)
- D) None

Question 8

Predict the output:

```
#include <stdio.h>
int main() {
    int x = -5;
    x = x >> 1;
    printf("%d\n", x);
    return 0;
}
```

- A) -2
- B) -3
- C) Error
- D) -1

Question 9

What is the output of the below code?

```
#include <stdio.h>
```

```
int main() {
    if (~0 == 1)
        printf("Yes");
    else
        printf("No");
    return 0;
}
```

- A) Yes
- B) No
- C) Error
- D) None

Question 10

What will the below code do?

```
void setBit(int* num, int pos) {
    *num |= (1 << pos);
}
```

- A) Divide num by pos
- B) Multiply num by pos
- C) Add num with pos
- D) Set the given bit position (pos) of number (num)

Question 11

What will be the output of the following code for N = 5 and K = 2?

```
bool function(int n, int k) {
    bool bit = n & (1 << k);
    return bit;
}
```

- A) 0
- B) 2
- C) 5
- D) 1

Question 12

By using which bitwise operator, we can find the **square of a number**?

- A) Bitwise AND (&)
- B) Bitwise OR (|)
- C) Both A and B
- D) Left Shift (<<) and Right Shift (>>)

Question 13

What does the below code do?

```
bool fun(int x, int y) {
    return ((x ^ y) < 0);
}
```

- A) Check if two numbers are even or odd
- B) Check if numbers are of same sign or not
- C) Check if numbers are multiples of 2
- D) None

Question 14

What does the below code do?

```
unsigned int function(int n) {
    unsigned int count = 0;
    while (n) {
        n &= (n - 1);
        count++;
    }
    return count;
}
```

- A) Finds the number of set bits in n
- B) Finds the count of 0's in n
- C) Both A and B
- D) None

Question 15

What will the below code snippet do?

```
const int x = 32;
char *function(char *a) {
    for (int i=0; a[i]!='\0'; i++)
        a[i] = a[i] & ~x;
    return a;
}
```

- A) Convert characters into integers
- B) Convert characters into binary
- C) Convert characters into uppercase letters
- D) All of these

Question 16

What is the return value of following function for `arr[] = {9, 12, 2, 11, 2, 2, 10, 9, 12, 10, 9, 11, 2}`?

```
int fun(int arr[], int n) {
    int x = arr[0];
    for (int i = 1; i < n; i++)
        x = x ^ arr[i];
    return x;
}
```

- A) 0
- B) 9
- C) 12
- D) 2

Question 17

Suppose we have a set {1, 2, 3}. Find the XOR of all subsets of the set.

- A) 1
- B) 2
- C) 3
- D) 0

Question 18

Which of the following is a **Single Operand Operator**?

- A) & Bitwise AND
- B) ~ Bitwise Negate
- C) ^ Bitwise XOR
- D) | Bitwise OR

Question 19

What will the following code do?

```
void bin(unsigned n) {
    if (n > 1)
        bin(n / 2);
    printf("%d", n % 2);
}
```

- A) Reverse the bits of a number
- B) Invert the bits of a number
- C) Print the binary representation of a number
- D) None

Question 20

What will be the output of the following code for $N = 128$?

```
#include <stdbool.h>
bool isPowerOfTwo(unsigned n) {
    return (n > 0) && ((n & (n - 1)) == 0);
}
int function(unsigned n) {
    if (!isPowerOfTwo(n))
        return -1;
    unsigned i = 1, pos = 1;
```

```

        while (!(i & n)) {
            i = i << 1;
            ++pos;
        }
        return pos;
}

```

- A) -1
- B) 1
- C) 64
- D) 8

Question 21

What condition should be applied in the below code snippet to check whether a number is even or odd using Bitwise Operator?

```

// C program to check for even or odd
#include <stdio.h>

// Returns 1 if n is even, else odd
int isEven(int n)
{
    if (_____)
        return 0;
    return 1;
}

int main()
{
    int n = 10;
    isEven(n) ? printf("Even") : printf("Odd");
    return 0;
}

```

- A) $n \mid 2$
- B) $n \& 1$
- C) $n \gg 1$
- D) $n \ll 1$

Question 22

What will the below code snippet do?

```

#include <stdio.h>
int main() {
    int num = 5;
    printf("%d\n", (~num + 1));
    return 0;
}

```

- A) It will print 2's complement of a number
- B) It will print 101
- C) It will print 1's complement of a number
- D) None

Question 23

If we want to invert all the bits of a number, then which bitwise operator should be used?

- A) ~
- B) &
- C) ^
- D) |

Question 24

You are given a list of 5 integers in the range 1 to 6. There are no duplicates, one integer is missing. Which of the following expression would give the missing number? (^ is XOR, ~ is NOT)

Let list elements be: list[0], list[1], list[2], list[3], list[4].

- A) list[0] ^ list[1] ^ list[2] ^ list[3] ^ list[4]
- B) list[0] ^ list[1] ^ list[2] ^ list[3] ^ list[4] ^ 1 ^ 2 ^ 3 ^ 4 ^ 5 ^ 6
- C) list[0] ^ list[1] ^ list[2] ^ list[3] ^ list[4] ^ 1 ^ 2 ^ 3 ^ 4 ^ 5
- D) ~(list[0] ^ list[1] ^ list[2] ^ list[3] ^ list[4])

Question 25

Right shift (>>) and Left shift (<<) are equivalent to _____ by 2.

- A) Multiply and Divide
- B) Divide and Multiply
- C) Addition and Subtraction
- D) Subtraction and Addition

Question 26

What will the following function do in bit manipulation?

```
int function(int n) {
    if (n % 4 == 0)
        return n;
    if (n % 4 == 1)
        return 1;
    if (n % 4 == 2)
        return n + 1;
    else
        return 0;
}
```

- A) Return the last set bit in a number
- B) Return the first set bit in a number
- C) XOR of two numbers
- D) XOR of numbers from 1 to N

Question 27

What is the output of the following code snippet?

```
#include <stdio.h>
void fun(int* num, int k) { *num &= (~(1 << k)); }

int main() {
    int num = 7;
    int k = 1;
    fun(&num, k);
    printf("%d\n", num);
    return 0;
}
```

- A) It will unset all bits of `num`
- B) It will clear all the bits of `bits`
- C) It will unset the `k`th bit of `num`
- D) None

Question 28

Consider the following function to check whether a number is a power of 2:

```
bool isPowerOfTwo (unsigned int x) {
    return (!(x & (x - 1)));
}
```

What is wrong with the above function?

- A) It does reverse of what is required
- B) It works perfectly fine for all values of `x`
- C) It does not work for `x = 0`
- D) It does not work for `x = 1`

Question 29

What does the following expression do?

```
x = x & (x - 1)
```

- A) Sets all bits as 1
- B) Makes `x` equal to 0
- C) Turns off the rightmost set bit
- D) Turns off the leftmost set bit

Question 30

What does the following C expression do?

```
x = (x << 1) + x + (x >> 1);
```


- A) Multiplies an integer with 7
- B) Multiplies an integer with 3.5
- C) Multiplies an integer with 3
- D) Multiplies an integer with 8